POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Project report

SWIM: An Efficient Framework to Train Neural Networks

Scientific computing tools for advanced mathematical modelling

Authors: Arianna Cagali, Mattia Gastoldi and Roberto Gastoldi

Academic year: 2024-2025

---

1. Introduction to SWIM

Sampling Where It Matters (**SWIM**) is a recently proposed method for training fully-connected neural networks without iterative optimization [1]. Instead of relying on gradient-based methods, SWIM **directly samples the weights and biases** of the network using both input and output data. The key idea is to construct weights based on pairs of data points that are informative, i.e., points that induce strong variations in the target function, as shown in Figure 1. This results in a data-driven sampling strategy that places activation regions of neurons where the function changes most, thus improving learning efficiency.

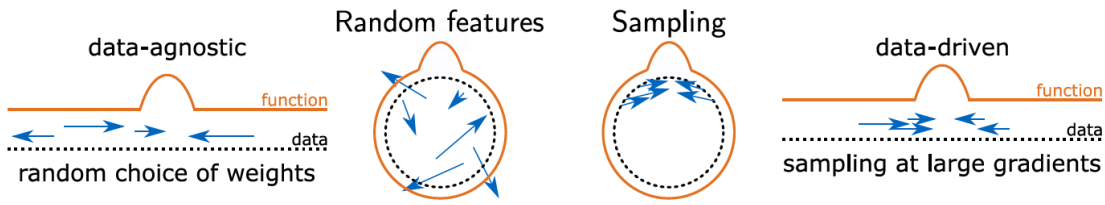


Figure 1: Random feature models choose weights in a data-agnostic way, compared to sampling them where it matters (SWIM): at large gradients. The arrows illustrate where the network weights are placed. The image is sourced from [1]

In contrast to classical random feature models, where parameters are sampled from data-agnostic distributions such as Gaussians or Uniform, SWIM defines weights through normalized differences between data pairs and sets biases accordingly. This enables the network to capture essential function features early in the forward pass, **without requiring backpropagation**. The only optimization step occurs in the final linear output layer, solved via a simple least-squares problem.
SWIM offers several advantages: it significantly reduces training time, often by orders of magnitude; it produces competitive results in terms of accuracy, especially in supervised settings with rich datasets; and it enhances interpretability, as each neuron's weight is explicitly linked to a specific data pair. Moreover, unlike many random feature methods, SWIM scales well to deep architectures, making it suitable for a wide range of problems, including operator learning and function regression.
However, SWIM has limitations. Its architecture is currently limited to fully-connected networks and cannot be easily extended to more complex structures such as convolutional neural netowrks (CNNs) or Transformer models. Additionally, in settings with very limited data, such as sparse measurements of PDE solutions, SWIM's performance decreases significantly, as it relies heavily on representative and abundant training data.

The goal of this project is to integrate the SWIM method into new test cases of particular interest that require the training of a neural network, in order to compare its effectiveness and its performance against more traditional approaches.

2. Mathematical formulation

In a sampled neural network, given a pair of points $(x^{(1)}, x^{(2)})$, the weight and bias of each neuron in layer ℓ are computed as follows:

$$w_{\ell,i} = s_1 \cdot \frac{x^{(2)}_{\ell-1,i} - x^{(1)}_{\ell-1,i}}{\left\| x^{(2)}_{\ell-1,i} - x^{(1)}_{\ell-1,i} \right\|^2}, \qquad b_{\ell,i} = \left\langle w_{\ell,i}, x^{(1)}_{\ell-1,i} \right\rangle + s_2.$$

This construction aligns each neuron's activation with directions in the input space where the target function exhibits the greatest variation. By defining the weight vector as the normalized difference between two informative data points, the neuron becomes sensitive to changes in the target and activates in the most relevant regions.
The bias is selected so that the activation boundary of the neuron (i.e., the associated hyperplane) passes through one of the selected points. This ensures that the neuron is centered around meaningful areas of the input space, enhancing the expressiveness of the network without requiring iterative training.
For the final (linear) output layer, the parameters are determined by solving a least-squares optimization problem:

$$(W_{L+1}, b_{L+1}) = \arg\min_{W,b} \left\| W \cdot \Phi^{(L)}(X) + b - Y \right\|_2^2$$

This step ensures that the final output best fits the target values in a standard regression sense.

3. Methods

For computational reasons, only a random subset of data pairs is considered. In each layer, an importance score is computed for every pair; this score is proportional to the normalized variation in the target variables and defines a probability distribution. According to this distribution, pairs of points are sampled, and the corresponding weights and biases are then computed. Finally, for the last layer, a least-squares optimization is employed to estimate the remaining parameters.
The detailed procedure is described in Algorithm 1.

Algorithm 1 SWIM: Sample Where It Matters

Require: • Data points $X = \{x_i \in \mathbb{R}^D\}_{i=1}^M$, targets $Y = \{y_i = f(x_i) \in \mathbb{R}^{N_{L+1}}\}_{i=1}^M$
• Number of layers L; Layer widths $N_1, \dots, N_{L+1}$; Activation function φ
• Constants $s_1, s_2 \in \mathbb{R}$; Regularization parameter $\varepsilon > 0$; Oversampling factor $\varsigma \in \mathbb{N}$

Initialize: $\Phi^{(0)}(x) \leftarrow x$
for $\ell = 1$ to L **do**
  Compute number of pairs: $M' \leftarrow \varsigma \cdot \left\lceil \frac{N_\ell}{M} \right\rceil \cdot M$
  Sample M' pairs $(x_i^{(1)}, x_i^{(2)})$ uniformly from $X \times X$
  Discard pairs where $\Phi^{(\ell-1)}(x_i^{(1)}) = \Phi^{(\ell-1)}(x_i^{(2)})$
  for each valid pair i **do**
    $z_i^{(1)} \leftarrow \Phi^{(\ell-1)}(x_i^{(1)}), \quad z_i^{(2)} \leftarrow \Phi^{(\ell-1)}(x_i^{(2)})$
    $y_i^{(1)} \leftarrow f(x_i^{(1)}), \quad y_i^{(2)} \leftarrow f(x_i^{(2)})$
    Compute importance:

$$P_i^{(\ell)} \leftarrow \frac{\|y_i^{(2)} - y_i^{(1)}\|}{\max\{\|z_i^{(2)} - z_i^{(1)}\|, \varepsilon\}}$$

  end for
  for $k = 1$ to N_ℓ **do**
    Sample a pair $(x^{(1)}, x^{(2)})$ with probability $\propto P^{(\ell)}$
    $z^{(1)} \leftarrow \Phi^{(\ell-1)}(x^{(1)}), \quad z^{(2)} \leftarrow \Phi^{(\ell-1)}(x^{(2)})$
    Compute weight:

$$w_{\ell,k} \leftarrow s_1 \cdot \frac{z^{(2)} - z^{(1)}}{\|z^{(2)} - z^{(1)}\|_2}$$

    Compute bias:

$$b_{\ell,k} \leftarrow \langle w_{\ell,k}, z^{(1)} \rangle + s_2$$

  end for
  Define layer output:

$$\Phi^{(\ell)}(x) \leftarrow \varphi(W_\ell \cdot \Phi^{(\ell-1)}(x) - b_\ell)$$

end for
Final Output Layer: Solve linear least-squares:

$$(W_{L+1}, b_{L+1}) \leftarrow \arg\min_{W,b} \|W \cdot \Phi^{(L)}(X) - Y + b\|_2^2$$

return All weights and biases $\{(W_\ell, b_\ell)\}_{\ell=1}^{L+1}$

4. Numerical results

Promising results with respect several benchmark problems have been discussed by the authors in [1], where SWIM proves his effectiveness also in more complicated settings. This section provides a more detailed analysis of SWIM's performance by applying this strategy to other case studies that typically require the training of neural networks. The results are evaluated through a direct comparison in terms of accuracy and training time between the sampling strategy adopted by SWIM and a conventional training approach based on gradient-based optimization.

4.1. Multi-Fidelity 2-Step Network

The first extension considered concerns a particular architecture: the *Multi-Fidelity 2-Step network*, first proposed in [4]. This architecture consists of two neural networks: the first is a fixed-structure network that maps an input to its corresponding low-fidelity (LF) function, referred to as NN_{LF}. The second network, denoted as NN_{HF}, has a single hidden layer with a variable number of neurons and maps an input together with its LF

output to the high-fidelity (HF) function. A schematic reppresentation of the network is shown in Figure 2.

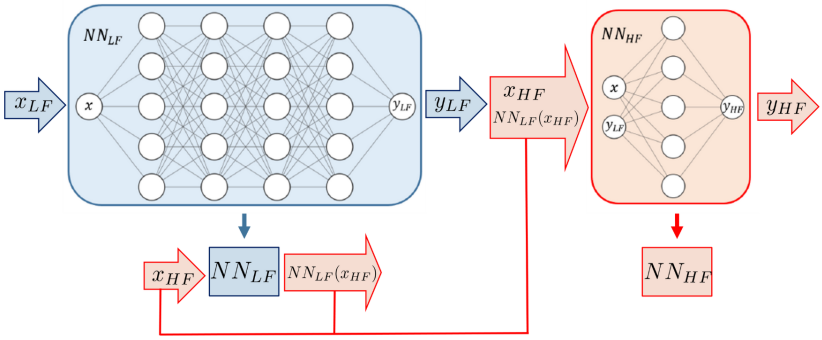


Figure 2: Multi-Fidelity 2-Step Network.

We focused on one of the examples presented in Section 4.1 of [4], leveraging the authors' publicly available code [2] for a direct comparison using the same hyperparameters (specifically, the number of neurons in NN_{HF}). In particular, the problem we considered is a well-known benchmark in computational and multi-fidelity modeling: the *Forrester function*, a common test case for MF methods. The HF and LF functions are defined over the domain $\Omega = [0, 1]$ as:

$$f_{\mathrm{HF}}(x) = (6x - 2)^2 \sin(12x - 4), \qquad f_{\mathrm{LF}}(x) = 0.5\, f_{\mathrm{HF}}(x) + 10(x - 0.5) + 5.$$

The LF function is derived through a linear transformation of the HF function and the input variable x. This setup provides a suitable initial testbed for MF models. For this problem, HF and LF samples are given by 6 and 32 equally spaced points in Ω, respectively.

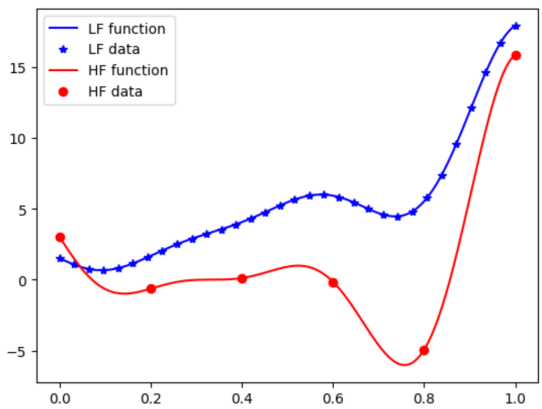
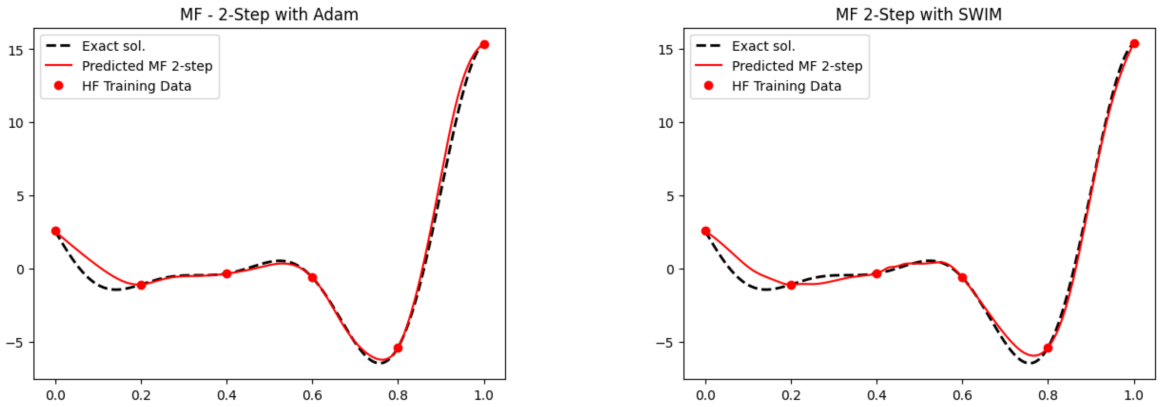


Figure 3: HF and LF reppresentation of the Forrester function.

The results of this extension[1] are shown in Figure 4. Unlike the original authors, in our case it was necessary to normalize the inputs before feeding them into the high-fidelity network ($\mathrm{NN}_{\mathrm{HF}}$). This was due to the fact that the two inputs (x, y_{LF}) belonged to domains with significantly different magnitudes, approximately $[0, 1] \times [-5, 15]$. Without normalization, the SWIM method's importance weights P would entirely suppress the contribution of the first input, effectively discarding meaningful spatial information.
In particular, the SWIM method proved to be extremely efficient even with such a simple architecture, making the training of the two networks nearly instantaneous, compared to the standard training procedure which

[1]Experiments were executed on a laptop with an Intel Core i5 dual-core CPU, 8 GB RAM, running macOS Ventura 13.7.6 and Python 3.11.11.

takes approximately 55 seconds. However, this gain in terms of training time is balanced by a slight reduction in reconstruction accuracy, with the relative error being almost 2% higher. Nevertheless, this trade-off is well justified given the overall performance of the method. The architectures used are summarized in Table 1 and Table 2.



(a) Training time: $56s$
Error: 9.81%.

(b) Training time: $0.0s$
Error: 11.16%.

Figure 4

Component LF_{NN}	**Specification**
Input $\rightarrow$ Output dimension	$1 \rightarrow 1$
Hidden layers	3 layers: [64, 64, 64] neurons
Activation functions (hidden layers)	`tanh`
Activation function (last layers)	`linear`
Training	Adam, $\eta = 0.0255$, Epochs $= 500$
Component HF_{NN}	**Specification**
Input $\rightarrow$ Output dimension	$2 \rightarrow 1$
Hidden layers	1 layers: 32 neurons
Activation functions (hidden layer)	`tanh`
Activation function (last layers)	`linear`
Training	Adam, $\eta = 0.0255$, Epochs $= 500$

Table 1: Summary of the involved MF 2-Step architecture.

Component LF_{NN}	Specification
Input → Output dimension	$1 \to 1$
Hidden layers	3 layers: [64, 64, 64] neurons
Activation functions (hidden layers)	`tanh`
Activation function (last layers)	`linear`
Component HF_{NN}	**Specification**
Input → Output dimension	$2 \to 1$
Hidden layers	1 layers: 32 neurons
Activation functions (hidden layer)	`tanh`
Activation function (last layers)	`linear`

Table 2: Summary of the involved MF 2-Step architecture with SWIM.

4.2. Solving PDEs

SWIM, in its standard formulation, proves to be a suboptimal choice for solving partial differential equations (PDEs), particularly when only limited training data are available. This behavior can be attributed to the fact that, although a data-driven sampling strategy is used for weights and biases, incorporating physical information into the network is not straightforward.
To leverage the advantages of SWIM in the context of physics-informed problems, the authors have introduced a variant designed to incorporate physical laws directly into the learning framework: **SWIM-PDE** [3].
The architecture used in SWIM-PDE follows a two-part structure, introducing a new layer type, the *Ansatz* Layer. That is, a dense neural network layer constructed using a sampling strategy for weights and biases (e.g. SWIM), using a differentiable activation function (e.g., `tanh`) and supporting differentiable operations such as gradients and Laplacians, making it suitable for computing differential operators.
The core idea is to define a neural approximation $\Phi(x,t)$ to the solution of a PDE, where the network is constrained not just by data but also by the governing equation.
Given a PDE of the general form:

$$\mathcal{L}(u, \partial u, \partial^2 u, \ldots; x, t) = f(x,t),$$

the training process is described in Algorithm 2.

Algorithm 2 SWIM-PDE training

Require: Measured data $\{(x_{\text{meas}}, u_{\text{meas}})\}$, PDE operator $\mathcal{L}(\cdot)$, forcing term $f(x,t)$, collocation points x_{col}, activation function φ, constants s_1, s_2, SWIM parameters

1: **Fit Ansatz Layer:** Construct network $\Phi(x,t)$ using the SWIM procedure on $(x_{\text{meas}}, u_{\text{meas}})$
2: **Evaluate Collocation Points:** Compute $\Phi(x_{\text{col}})$ and evaluate $\mathcal{L}(\Phi(x_{\text{col}}))$ using automatic differentiation
3: **Evaluate Forcing Term:** Compute $f(x_{\text{col}})$ at collocation points
4: **Solve Least Squares Problem:** Solve the system:

$$\begin{bmatrix} u_{\text{meas}} \\ f(x_{\text{col}}) \end{bmatrix} = \begin{bmatrix} \Phi(x_{\text{meas}}) \\ \mathcal{L}(\Phi(x_{\text{col}})) \end{bmatrix} \beta$$

to find the optimal parameters β of the final linear layer
5: **return** Trained model $u(x,t) \approx \Phi(x,t)^T \beta$

4.2.1 SWIM-PDE for the Eikonal problem

Mathematical formulation. The spatial propagation of a heartbeat can be described by the Eikonal model:

$$C_v\sqrt{\nabla T(x,y)^\top D\nabla T(x,y)} = 1, \quad (x,y)\in\Omega, \qquad T(x_0,y_0)=t_0, \quad T(x_1,y_1)=t_1, \tag{1}$$

where:

$$D = \vec{b}\otimes\vec{b} + \frac{1}{a_{\text{ratio}}}, \vec{a}\otimes\vec{a},$$

with $\vec{b}$ denoting the fiber orientation and $\vec{a}$ the orthogonal cross-fiber direction. The function $T(x,y)$ represents the arrival time of the electrical signal at a given location, i.e., the moment the heartbeat reaches that point. Given the physical parameters of the Eikonal equation in (1), the goal is to reconstruct the field $T(x,y)$ given a significant number of measurements in the space.

Methods. SWIM and SWIM-PDE require a significant number of observations to achieve sufficient accuracy. In this application, the aim is to assess whether SWIM-PDE can effectively reconstruct the activation time field $T(x,y)$, governed by a complex PDE such as the Eikonal equation, even in the absence of boundary conditions, provided that a much larger number of measurements is available.
A total of 300 measurement points are considered. Unlike in real-world application, the measurements sampled directly from the ground truth solution, with added noise. A SWIM-PDE training procedure is then applied. The architecture employed is summarized in Table 3.

Component	Specification
Input $\to$ Output dimension	$2 \to 1$
Ansatz layer	800 basis
Activation function	`tanh`
Collocation points	900

Table 3: Summary of the SWIM-PDE architecture with 300 measurements.

Results. The final field reconstruction is reported in Figure 5, and its average training time[2] in Table 4.

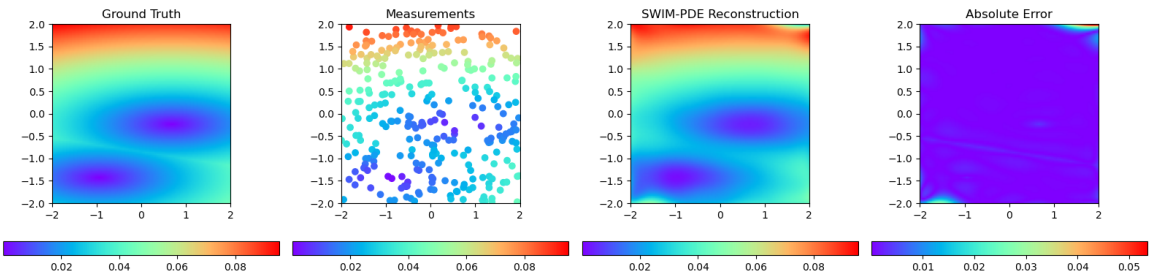


Figure 5: SWIM-PDE reconstruction and absolute error.

Method	Time (s)
SWIM-PDE	1.023

Table 4: Average training time of SWIM-PDE for the reconstruction.

4.2.2 SWIM-PDE for the Eikonal problem: comparison with PINN

Given the physical parameters of the Eikonal equation in (1), the goal is to reconstruct the field $T(x,y)$ starting from 20 activation times measured by a star-shaped catheter.

[2]Executed on a laptop equipped with an Intel Core i5-1135G7 CPU (4 cores), 8 GB RAM, Windows 11 Pro 64-bit, and Python 3.11.11.

Methods. To solve PDEs using Neural Networks, Physics-Informed Neural Networks (PINNs) are considered the gold standard. Thanks to their flexibility in leveraging both physical laws and measurement data, they can provide accurate solutions to complex differential problems. However, PINNs present two main limitations: they require extensive hyperparameter tuning, and the online training procedure used in recursive estimation is computationally expensive. Neural networks trained with SWIM (and its PDE-specific variant) offer an efficient alternative with fewer hyperparameters to fine-tune. The goal is to assess whether training with SWIM can serve as a viable alternative to PINNs, specifically in the considered Eikonal problem, where few measurements are collected, like in a real-world scenario, and no boundary conditions are available.
The architecture of the fine-tuned Physics-Informed and SWIM-PDE neural networks are summarized in Table 5 and Table 6 respectively.

Component	**Specification**
Input $\rightarrow$ Output dimension	$2 \rightarrow 1$
Hidden layers	2 layers: [8, 16] neurons
Activation functions (all layers)	`tanh`
Training	Adam, $\eta = 0.003$, Epochs $= 40000$
Loss Function Coefficient	**Value**
λ_{smooth} (Smallest collocation point)	1.0
λ_{reg} (400 boundary points)	$5e-1$
λ_{phy} (4000 collocation points)	$1e-4$
λ_{data} (20 measurements)	1.0

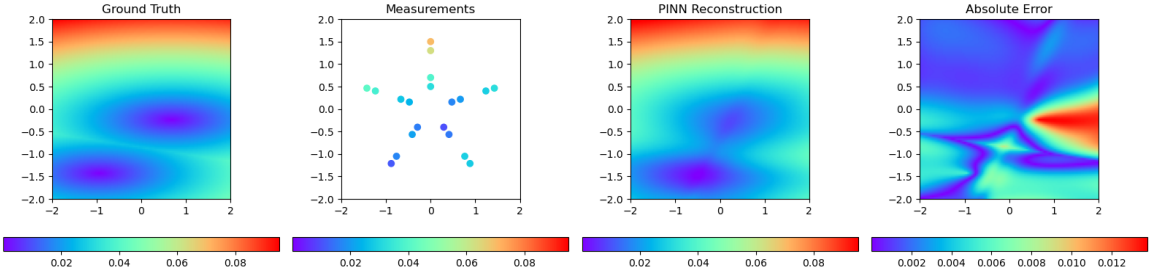
Table 5: Summary of the PINN architecture.

Component	**Specification**
Input $\rightarrow$ Output dimension	$2 \rightarrow 1$
Ansatz layer	800 basis
Activation function	`tanh`
Collocation points	16

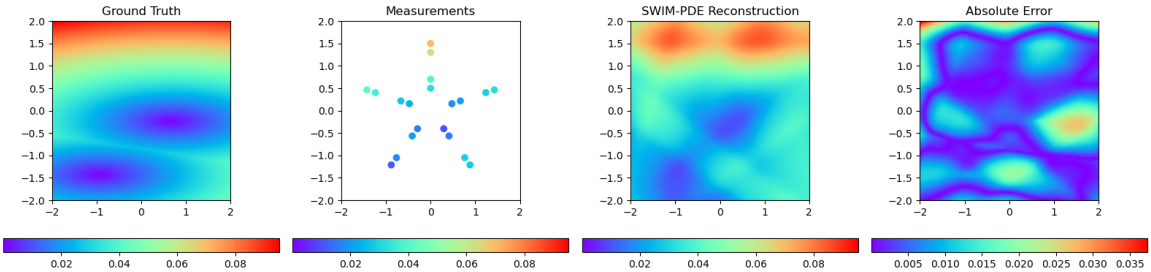
Table 6: Summary of the SWIM-PDE architecture.

Results. The performances in terms of accuracy and training time[3] are reported in Figure 6, and Table 7 respectively.

[3]Executed on a laptop equipped with an Intel Core i5-1135G7 CPU (4 cores), 8 GB RAM, Windows 11 Pro 64-bit, and Python 3.11.11.



(a) PINN reconstruction and absolute error.



(b) SWIM-PDE reconstruction and absolute error.

Figure 6: Activation time field reconstruction with (a) PINN and (b) SWIM-PDE given the 20 measurements.

Method	Time (s)
PINN	102
SWIM-PDE	0.021

Table 7: Average training time for PINN and SWIM-PDE.

4.2.3 General comments on the overall performances of SWIM-PDE

The SWIM-PDE variant builds upon the same principles as the original SWIM algorithm, inheriting its key advantages while adapting them to the context of PDEs resolution, integrating physics in the architecture.
However, the method proves to be highly effective in terms of both accuracy and efficiency only when a large number of measurements is available, even in the absence of boundary conditions. In scenarios with limited measurements, and under the same conditions, Physics-Informed Neural Networks (PINNs) show superior accuracy and flexibility, despite being computationally more expensive. This advantage stems from the ability to regularize the solution explicitly through the loss function. Still, SWIM-PDE is capable of providing approximately reasonable solutions even with few data points. Therefore, an interesting strategy could be to combine the two approaches: using SWIM-PDE to provide an effective initial guess for standard PINN training.
It is important to note the strong dependence of SWIM-PDE performance on two tunable parameters: the number of neurons in the Ansatz layer and the number of collocation points. Compared to a generic network, the Ansatz layer typically requires a higher number of neurons to better capture the complexity of the solution. Regarding the collocation points, their selection is highly dependent on the number of measurements available. Indeed, the number of collocation points plays a role similar to the physics-penalty parameter in the loss function of a PINN. A high number of collocation points may overwhelm the influence of the measurements, while a low number might fail to adequately incorporate the physical knowledge. A reasonable recommendation is to choose a number of collocation points comparable to the number of measurements.

5. Conclusions

SWIM offers significant efficiency gains by eliminating the need for gradient-based optimization, resulting in much faster training times compared to traditional methods. It maintains high accuracy, performing comparably to iteratively trained networks on various tasks involving fully connected architectures. Moreover, it enhances

interpretability, as each weight and bias is explicitly derived from specific pairs of training data, making the internal logic of the model more transparent.
The benefits of this approach extend to concatenated neural networks, such as in multi-fidelity learning, where SWIM achieves competitive accuracy while drastically reducing training time. However, the method currently remains limited in scope and is not directly applicable to more complex architectures.
The SWIM-PDE variant extends the SWIM algorithm to the resolution of PDEs by embedding physical knowledge directly into the network architecture, thus avoiding recursive training and significantly reducing computational costs. While SWIM-PDE provides fast and reasonably accurate solutions, its effectiveness relies on the availability of a large number of measurements. In low-data regimes, PINNs tend to outperform SWIM-PDE due to their flexibility. Moreover, SWIM-PDE performance depends strongly on the number of neurons in the Ansatz layer and the number of collocation points. In particular, the second one acts similarly to the physics-weight term in PINN training. A good choice is to set the number of collocation points roughly equal to the number of measurements.

References

[1] Erik L Bolager, Iryna Burak, Chinmay Datar, Qing Sun, and Felix Dietrich. Sampling weights of deep neural networks. *Advances in Neural Information Processing Systems*, 36:63075–63116, 2023.

[2] Paolo Conti. `https://github.com/ContiPaolo/MultiFidelity_NNs`.

[3] Chinmay Datar, Taniya Kapoor, Abhishek Chandra, Qing Sun, Iryna Burak, Erik Lien Bolager, Anna Veselovska, Massimo Fornasier, and Felix Dietrich. Solving partial differential equations with sampled neural networks. *arXiv preprint arXiv:2405.20836*, 2024.

[4] Mengwu Guo, Andrea Manzoni, Maurice Amendt, Paolo Conti, and Jan S Hesthaven. Multi-fidelity regression using artificial neural networks: Efficient approximation of parameter-dependent output quantities. *Computer methods in applied mechanics and engineering*, 389:114378, 2022.